



خلاصه

این آزمایش سه بخش دارد: پیمانه‌ی جمع و تفریق اعداد مختلط، پیمانه‌ی ضرب اعداد مختلط، و یک واحد پایپ‌لاین که دستورات را از یک حافظه‌ی ۳۲ کلمه‌ای می‌خواند و عملیات مختلط را روی آن‌ها انجام می‌دهد.

قید کوتاهی در انتهای صورت آزمایش آمده که در عمل تمام معماری را تعیین می‌کند:

«در این آزمایش، نایستی از واحدهای جمع کننده یا ضرب کننده بیش از یک بار استفاده شود.»

یعنی در کل ماشین یک جمع‌کننده و یک ضرب‌کننده وجود دارد. تمام تصمیم‌هایی که در ادامه می‌آید — یکی بودن ماژول جمع/تفریق و ضرب، زمان‌بندی چهار مرحله‌ای ضرب، و اینکه مرحله‌ی اجرای پایپ‌لاین چند کلاکی است — پیامد مستقیم همین یک جمله است.

نمایش عدد مختلط

هر عدد مختلط یک زوج از دو مؤلفه‌ی علامت‌دار مکمل ۲ هشت‌بیتی است که در یک کلمه‌ی ۱۶ بیتی بسته‌بندی می‌شود:

$$\text{word} = \{ \text{imag}[7:0], \text{real}[7:0] \} \quad \text{بازه‌ی هر مؤلفه: } ۱۲۸ - \text{تا } ۱۲۷ +$$

نتایج اشباع می‌شوند نه سرریز: مؤلفه‌ای که از بازه بیرون بزند به ۱۲۸- یا ۱۲۷+ چسبانده می‌شود و پرچمی بالا می‌رود. برای ماشینی که کارش حساب روی کمیت‌های اندازه‌گیری‌شده است این رفتار مفیدتر است — مقداری که خیلی بزرگ شده باید «خیلی بزرگ» خوانده شود، نه اینکه بی‌سروصدا علامتش عوض شود. مثلاً $۱۰۰۰ = ۱۰۰ \times ۱۰۰$ با سرریز عادی به ۱۶ تبدیل می‌شد که هیچ ربطی به پاسخ درست ندارد.

چرا جمع/تفریق و ضرب یک ماژول‌اند

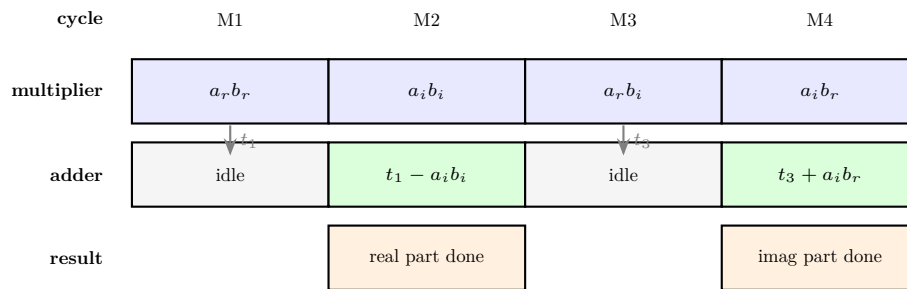
ضرب دو عدد مختلط به چهار ضرب حقیقی و دو جمع حقیقی نیاز دارد:

$$(a_r + j a_i)(b_r + j b_i) = (a_r b_r - a_i b_i) + j (a_r b_i + a_i b_r)$$

و جمع دو عدد مختلط به دو جمع حقیقی دیگر. اگر بخش الف و بخش ب دو ماژول جدا می‌بودند، سطح بالای ماشین دو جمع‌کننده می‌داشت — دقیقاً همان چیزی که قید بالا ممنوع کرده است.

بنابراین هر دو عملیات در یک ماژول به نام `complex_alu` زندگی می‌کنند که داخل آن دقیقاً یک `mul_unit` و یک `add_unit` قرار دارد. بخش الف و بخش ب دو حالت کاری این ماژول‌اند و هر کدام تست‌بنچ مستقل خودشان را دارند. بهایی که برای این قید پرداخت می‌شود در زمان است نه در سطح: ماژول به‌جای ترکیبی، ترتیبی است.

زمان‌بندی واحد مشترک



نکته‌ی اصلی این زمان‌بندی در ستون‌های M2 و M4 است: جمع‌کننده روی حاصل‌ضربی کار می‌کند که در همان کلاک از ضرب‌کننده بیرون می‌آید، در حالی که ضرب‌کننده هم بی‌کار نمانده و مشغول حاصل‌ضرب بعدی است. بدون این هم‌پوشانی، ضرب مختلط ۴ کلاک ضرب به علاوه‌ی ۲ کلاک جمع، یعنی ۶ کلاک، طول می‌کشید. با آن، ۴ کلاک — و هیچ واحد اضافه‌ای هم به مدار اضافه نشده است.

عملیات	کلاک اشغال واحد مشترک	علت
ADD / SUB	۲	دو جمع حقیقی، یک جمع‌کننده
MUL	۴	چهار ضرب حقیقی، یک ضرب‌کننده

هر دو عدد در تست‌بنچ‌ها صریحاً بررسی شده‌اند، نه فقط نتیجه‌ی عددی؛ چون درست بودن پاسخ به‌تنهایی نشان نمی‌دهد که مدار واقعا از یک واحد استفاده کرده باشد.

```

1  `include "complex_pkg.vh"
2
3  // Parts (a) and (b) of the lab: complex add/subtract and complex multiply.
4  //
5  // Why they are ONE module
6  // -----
7  // The lab sheet says no adder or multiplier unit may be used more than once in
8  // the finished machine. Complex multiplication needs four real products and two
9  // real additions:
10 //
11 //      (ar + j*ai)(br + j*bi) = (ar*br - ai*bi) + j(ar*bi + ai*br)
12 //
13 // and complex addition needs two more real additions. If add/subtract and
14 // multiply were separate modules, the top level would contain two adders --
15 // exactly what the constraint forbids. So both operations live here, sharing
16 // one mul_unit and one add_unit, and the price of the constraint is paid in
17 // TIME instead of area: the module is sequential.
18 //
19 // Schedule
20 // -----
21 //   ADD / SUB      A_RE:  real parts                2 cycles of work
22 //                  A_IM:  imaginary parts
23 //
24 //   MUL            M1: mul = ar*br                  4 cycles of work
25 //                  M2: mul = ai*bi , add = t1 - mul  (real part done)
26 //                  M3: mul = ar*bi
27 //                  M4: mul = ai*br , add = t3 + mul  (imag part done)
28 //
29 // Note how M2 and M4 use the adder on the product being computed in that very
30 // cycle while the multiplier is already busy with the next one. That overlap
31 // is what brings the multiply down from 6 cycles to 4 without adding a second
32 // unit of either kind.
33 module complex_alu (
34     input wire      Clk,
35     input wire      RstN,
36     input wire      start,
37     input wire [1:0] op,          // ALU_ADD / ALU_SUB / ALU_MUL
38     input wire [CX_WORD-1:0] a,    // { imag, real }
39     input wire [CX_WORD-1:0] b,
40     output reg [CX_WORD-1:0] y,
41     output reg      ovf,          // a component had to be clamped
42     output wire     busy,
43     output reg      done          // 1 clk pulse, y valid in that cycle
44 );

```

```

45 localparam [2:0] IDLE = 3'd0,
46                A_RE = 3'd1, A_IM = 3'd2,
47                M1 = 3'd3, M2 = 3'd4, M3 = 3'd5, M4 = 3'd6;
48
49 reg [2:0] st;
50 reg [`CX_WORD-1:0] a_r, b_r; // operands latched at start
51 reg [1:0] op_r;
52 reg signed [15:0] t1, t3; // latched partial products
53 reg sat_r; // real part had to be clamped
54
55 wire signed [7:0] ar = a_r[7:0], ai = a_r[15:8];
56 wire signed [7:0] br = b_r[7:0], bi = b_r[15:8];
57
58 // ---- the one multiplier and the one adder ----
59 reg signed [7:0] mx, my;
60 wire signed [15:0] mp;
61 mul_unit u_mul (.x(mx), .y(my), .p(mp));
62
63 reg signed [16:0] ax, ay;
64 reg asub;
65 wire signed [16:0] as;
66 add_unit u_add (.x(ax), .y(ay), .sub(asub), .s(as));
67
68 assign busy = (st != IDLE);
69
70 // Clamp a 17-bit result into a signed 8-bit component.
71 function signed [7:0] sat8;
72     input signed [16:0] v;
73     begin
74         if (v > 17'sd127) sat8 = 8'sd127;
75         else if (v < -17'sd128) sat8 = -8'sd128;
76         else sat8 = v[7:0];
77     end
78 endfunction
79
80 function clipped;
81     input signed [16:0] v;
82     begin
83         clipped = (v > 17'sd127) || (v < -17'sd128);
84     end
85 endfunction
86
87 // ---- operand routing: which numbers reach the two shared units ----
88 always @(*) begin
89     mx = 8'sd0;
90     my = 8'sd0;
91     ax = 17'sd0;
92     ay = 17'sd0;
93     asub = 1'b0;
94     case (st)
95         A_RE: begin
96             ax = {{9{ar[7]}}, ar};
97             ay = {{9{br[7]}}, br};
98             asub = (op_r == `ALU_SUB);
99         end
100        A_IM: begin
101            ax = {{9{ai[7]}}, ai};
102            ay = {{9{bi[7]}}, bi};
103            asub = (op_r == `ALU_SUB);
104        end
105        M1: begin mx = ar; my = br; end
106        M2: begin
107            mx = ai; my = bi; // computing ai*bi ...
108            ax = {t1[15], t1}; // ... while the adder does
109            ay = {mp[15], mp}; // ar*br - ai*bi
110            asub = 1'b1;
111        end
112        M3: begin mx = ar; my = bi; end
113        M4: begin
114            mx = ai; my = br; // computing ai*br ...
115            ax = {t3[15], t3}; // ... while the adder does
116            ay = {mp[15], mp}; // ar*bi + ai*br
117            asub = 1'b0;
118        end
119        default: ;
120    endcase
121 end
122
123 always @(posedge Clk or negedge RstN) begin
124     if (!RstN) begin
125         st <= IDLE;
126         y <= {`CX_WORD{1'b0}};
127         ovf <= 1'b0;
128         done <= 1'b0;
129         a_r <= {`CX_WORD{1'b0}};
130         b_r <= {`CX_WORD{1'b0}};
131         op_r <= 2'd0;

```

```

132     t1    <= 16'sd0;
133     t3    <= 16'sd0;
134     sat_r <= 1'b0;
135 end else begin
136     done <= 1'b0;
137     case (st)
138     IDLE: if (start) begin
139         a_r    <= a;
140         b_r    <= b;
141         op_r    <= op;
142         sat_r <= 1'b0;
143         st     <= (op == `ALU_MUL) ? M1 : A_RE;
144     end
145
146     A_RE: begin
147         y[7:0] <= sat8(as);
148         sat_r <= clipped(as);
149         st     <= A_IM;
150     end
151     A_IM: begin
152         y[15:8] <= sat8(as);
153         ovf     <= sat_r | clipped(as);
154         done    <= 1'b1;
155         st     <= IDLE;
156     end
157
158     M1: begin t1 <= mp; st <= M2; end
159     M2: begin
160         y[7:0] <= sat8(as);
161         sat_r <= clipped(as);
162         st     <= M3;
163     end
164     M3: begin t3 <= mp; st <= M4; end
165     M4: begin
166         y[15:8] <= sat8(as);
167         ovf     <= sat_r | clipped(as);
168         done    <= 1'b1;
169         st     <= IDLE;
170     end
171
172     default: st <= IDLE;
173 endcase
174 end
175 end
176 endmodule

```

دو واحد مشترک خودشان کوچک‌اند. توجه کنید که تفریق واحد دومی نیست: $x - y$ همان جمع‌کننده با معکوس کردن y و یک رقم نقلی ورودی است، که دقیقاً همان چیزی است که سترکننده می‌سازد.

```

1 // THE multiplier of the machine.
2 //
3 // The lab sheet allows a single adder and a single multiplier in the whole
4 // design, so this module is instantiated exactly once -- inside complex_alu --
5 // and every product the machine ever computes passes through it, one per
6 // clock cycle.
7 module mul_unit (
8     input wire signed [7:0] x,
9     input wire signed [7:0] y,
10    output wire signed [15:0] p
11);
12    assign p = x * y;
13 endmodule

```

```

1 // THE adder/subtractor of the machine, instantiated exactly once.
2 //
3 // Width is 17 bits because the same unit has to serve two very different
4 // jobs: adding two 8-bit components (9 bits would do) and combining two
5 // 16-bit partial products of a complex multiply (17 bits needed).
6 //
7 // Subtraction is not a second unit:  $x - y$  is the same adder with  $y$  inverted
8 // and a carry into the low bit, which is how a synthesiser implements it.
9 module add_unit (
10    input wire signed [16:0] x,
11    input wire signed [16:0] y,
12    input wire sub,
13    output wire signed [16:0] s
14);
15    assign s = sub ? (x - y) : (x + y);
16 endmodule

```

پهنای جمع‌کننده ۱۷ بیت است، چون همین یک واحد باید دو کار خیلی متفاوت را انجام دهد: جمع دو مؤلفه‌ی ۸ بیتی (که ۹ بیت کافی بود) و ترکیب دو حاصل ضرب جزئی ۱۶ بیتی (که ۱۷ بیت لازم دارد).

ماشین: مجموعه دستورات و حافظه

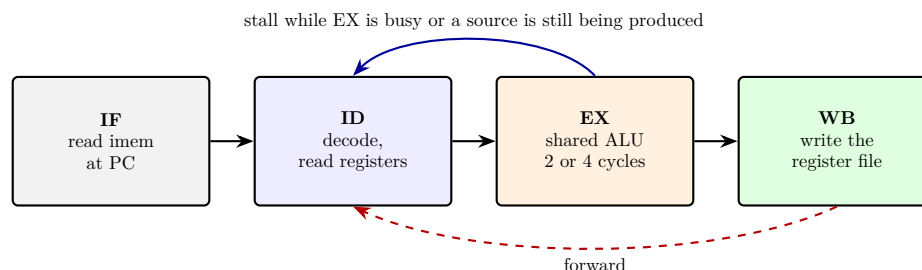
صورت آزمایش می‌گوید ماشین یک حافظه‌ی ۳۲ کلمه‌ای دارد و واحد پایپ‌لاین دستورات را از همان حافظه می‌خواند. هر کلمه ۳۲ بیت است:

opcode [31:28]	rd [27:25]	rs [24:22]	rt [21:19]	— [18:16]	immediate = { imag, real } [15:0]
-------------------	---------------	---------------	---------------	--------------	--------------------------------------

کد	دستور	اثر	توضیح
0	NOP	—	حباب پایپ‌لاین
1	LDI	$rd \leftarrow imm$	تنها راه وارد کردن ثابت
2	MOV	$rd \leftarrow rs$	بدون استفاده از ALU
3	ADD	$rd \leftarrow rs + rt$	دو کلاک جمع‌کننده
4	SUB	$rd \leftarrow rs - rt$	دو کلاک جمع‌کننده
5	MUL	$rd \leftarrow rs \times rt$	چهار کلاک ضرب‌کننده
F	HALT	—	واکشی را متوقف می‌کند

ثبات‌ها هشت‌تا و هرکدام یک عدد مختلط کامل‌اند. LDI و MOV عمداً از ALU استفاده نمی‌کنند؛ این باعث می‌شود در بخش تست بشود اثر خالص پایپ‌لاین را از اثر واحد مشترک جدا اندازه گرفت.

پایپ‌لاین



پیامد جالب قید تک‌واحدی این است که مرحله‌ی EX خودش نمی‌تواند پایپ‌لاین شود: وقتی فقط یک ضرب‌کننده وجود دارد، تنها یک حاصل‌ضرب می‌تواند در جریان باشد. اما IF و ID و WB همچنان با EX هم‌پوشانی دارند — در حالی که یک ضرب چهار کلاک طول می‌کشد، دستور بعدی واکشی و کدگشایی شده و نتیجه‌ی دستور قبلی در حال نوشته شدن است. به بیان دیگر EX گلوگاه گذردهی است، ولی این یک انتخاب اعلام‌شده است نه یک اشتباه.

دو قفل داخلی درستی را تضمین می‌کنند:

الف) قفل منبع. مرحله‌ی EX فقط وقتی دستور تازه می‌پذیرد که آزاد باشد (یا در همین کلاک آزاد شود). بنابراین یک عملیات طولانی، کدگشایی و واکشی را پشت خودش متوقف می‌کند.

ب) قفل داده و ارسال به جلو. دستوری که ثبات مبدأش هنوز در EX در حال تولید است، در ID منتظر می‌ماند. به محض اینکه تولیدکننده به WB برسد، مقدار مستقیماً به مرحله‌ی کدگشایی ارسال به جلو می‌شود. نتیجه این است که انتظار دقیقاً به اندازه‌ای است که باید باشد و نه یک کلاک بیشتر — عددی که در بخش تست اندازه‌گیری شده است.

ج) اعلام پایان کار. سیگنال halted فقط یک چیز می‌گوید: واکشی متوقف شده است. ولی در همان لحظه ممکن است آخرین دستور هنوز در EX باشد، یا نتیجه‌اش در ثبات WB منتظر لبه‌ی بعدی کلاک نشسته باشد. هر کسی که با دیدن halted فایل ثبات را بخواند، مقدار قدیمی می‌بیند.

به همین دلیل خروجی جداگانه‌ای به نام finished وجود دارد که علاوه بر توقف واکنشی، خالی بودن پایپ‌لاین را هم شرط می‌کند:

```
assign finished = halted && !ex_v && !wb_v;
```

این تمایز در نگاه اول بی‌اهمیت به نظر می‌رسد ولی دقیقاً همان چیزی است که در اولین نسخه‌ی تست‌بنج باعث خطا شد: تست با دیدن halted ثبات‌ها را می‌خواند و نتیجه‌ی آخرین دستور را از دست می‌داد. راه درست، اصلاح واسط بود نه اضافه کردن تاخیر به تست.

نکته‌ی ظریف دیگری که ارزش گفتن دارد: ارسال به جلو از WB به ID لازم است، نه یک بهینه‌سازی اختیاری. آرایه‌ی ثبات‌ها در لبه‌ی کلاک نوشته می‌شود و خواندن آن ترکیبی است، پس دستوری که در همان کلاک نوشتن، همان ثبات را می‌خواند مقدار قدیمی را می‌بیند. بدون این مسیر، برنامه‌های وابسته پاسخ غلط می‌دادند.

```
1  `include "complex_pkg.vh"
2
3  // Part (c) of the lab: the machine that reads instructions out of the 32-word
4  // memory and executes them in a pipeline.
5  //
6  // Stages
7  // -----
8  // IF read imem at PC, register the instruction
9  // ID decode, read the register file, resolve hazards
10 // EX the shared complex ALU -- 2 or 4 cycles of work, or one cycle for the
11 // instructions that need no arithmetic at all (LDI / MOV / NOP)
12 // WB write the result back
13 //
14 // The interesting consequence of the single-adder / single-multiplier rule is
15 // that EX cannot itself be pipelined: there is one multiplier, so only one
16 // product can be in flight at a time. IF, ID and WB still overlap with EX --
17 // while a multiply is grinding away, the next instruction is already fetched
18 // and decoded and the previous result is being written back -- but EX is the
19 // throughput limit by construction, not by accident.
20 //
21 // Two interlocks keep that correct:
22 // * EX accepts a new instruction only when it is free (or freeing this
23 // cycle), so a long operation stalls decode and fetch behind it;
24 // * an instruction whose source register is still being produced in EX waits
25 // in ID. Once the producer reaches WB the value is forwarded straight into
26 // the decode stage, so the wait is exactly as long as it has to be.
27 module complex_cpu (
28     input wire          Clk,
29     input wire          RstN,
30     input wire          run,          // 0 = hold (used while loading)
31
32     // program load port
33     input wire          prog_we,
34     input wire [31:0]   prog_addr,
35     input wire [31:0]   prog_data,
36
37     // observation
38     input wire [31:0]   dbg_raddr,
39     output wire [31:0]  dbg_rdata,
40     output reg [31:0]   pc,
41     output reg          halted,      // fetching has stopped
42     output wire         finished,    // ... and the pipeline is empty
43     output reg          ovf_sticky,
44     output wire         alu_busy,
45     output wire         ex_busy,
46     output wire         wb_we,
47     output wire [31:0]  wb_rd,
48     output wire [31:0]  wb_data
49 );
50 // ----- IF --
51 wire [31:0] if_instr;
52
53 imem u_imem (
54     .Clk(Clk), .we(prog_we), .waddr(prog_addr), .wdata(prog_data),
55     .raddr(pc), .rdata(if_instr)
56 );
57
58 // ----- IF / ID --
59 reg          id_v;
60 reg [31:0]   id_ir;
61
62 wire [3:0]    id_op = id_ir[31:28];
63 wire [31:0]  id_rd = id_ir[27:25];
64 wire [31:0]  id_rs = id_ir[24:22];
65 wire [31:0]  id_rt = id_ir[21:19];
66 wire [31:0]  id_imm = id_ir[15:0];
67
```

```

68 wire id_is_alu = (id_op == `OP_ADD) || (id_op == `OP_SUB) || (id_op == `OP_MUL);
69 wire id_is_halt = (id_op == `OP_HALT);
70 wire id_writes = id_is_alu || (id_op == `OP_LDI) || (id_op == `OP_MOV);
71 wire id_uses_rs = id_is_alu || (id_op == `OP_MOV);
72 wire id_uses_rt = id_is_alu;
73
74 wire [1:0] id_alu_op = (id_op == `OP_SUB) ? `ALU_SUB :
75                       (id_op == `OP_MUL) ? `ALU_MUL : `ALU_ADD;
76
77 // ----- ID --
78 wire [`CX_WORD-1:0] rf_rd1, rf_rd2;
79 reg                wb_v;
80 reg [`REG_AW-1:0] wb_a;
81 reg [`CX_WORD-1:0] wb_d;
82
83 assign wb_we = wb_v;
84 assign wb_rd = wb_a;
85 assign wb_data = wb_d;
86
87 regfile u_rf (
88     .Clk(Clk), .RstN(RstN),
89     .we(wb_v), .wa(wb_a), .wd(wb_d),
90     .ra1(id_rs), .ra2(id_rt), .rd1(rf_rd1), .rd2(rf_rd2),
91     .ra_dbg(dbg_raddr), .rd_dbg(dbg_rdata)
92 );
93
94 // Write-back forwarding: the register array is written on this edge, so a
95 // combinational read of the same address still returns the old value.
96 wire [`CX_WORD-1:0] rs_val = (wb_v && wb_a == id_rs) ? wb_d : rf_rd1;
97 wire [`CX_WORD-1:0] rt_val = (wb_v && wb_a == id_rt) ? wb_d : rf_rd2;
98
99 // ----- ID / EX --
100 reg                ex_v;
101 reg                ex_is_alu;
102 reg                ex_writes;
103 reg [`REG_AW-1:0] ex_rd;
104 reg [`CX_WORD-1:0] ex_simple; // result of LDI / MOV, which need no ALU
105
106 reg                alu_start;
107 reg [1:0]          alu_op;
108 reg [`CX_WORD-1:0] alu_a, alu_b;
109 wire [`CX_WORD-1:0] alu_y;
110 wire                alu_done, alu_ovf;
111
112 complex_alu u_alu (
113     .Clk(Clk), .RstN(RstN),
114     .start(alu_start), .op(alu_op), .a(alu_a), .b(alu_b),
115     .y(alu_y), .ovf(alu_ovf), .busy(alu_busy), .done(alu_done)
116 );
117
118 assign ex_busy = ex_v;
119
120 // `halted` only means the fetch stage stopped: the last instruction may
121 // still be in EX, and its result may still be sitting in the write-back
122 // register waiting for the next clock edge. Anything that reads the
123 // register file when the program ends must wait for `finished`, which
124 // additionally requires the pipeline to be empty. (wb_v high means the
125 // write lands on the coming edge, so it has to be clear too.)
126 assign finished = halted && !ex_v && !wb_v;
127
128 // ----- control --
129 wire ex_completing = ex_v && (!ex_is_alu || alu_done);
130 wire ex_can_accept = !ex_v || ex_completing;
131
132 // A source register still being produced in EX is not readable yet. The
133 // instruction waits in ID; once the producer moves on to WB the forward
134 // above supplies the value, so the stall lasts exactly one cycle longer
135 // than the producer's own execution.
136 wire hazard = ex_v && ex_writes &&
137             ((id_uses_rs && ex_rd == id_rs) ||
138              (id_uses_rt && ex_rd == id_rt));
139
140 wire halting = id_v && id_is_halt && ex_can_accept;
141 wire id_go = id_v && ex_can_accept && !hazard;
142 wire if_go = !halted && !halting && (!id_v || id_go);
143
144 // ----- sequential --
145 always @(posedge Clk or negedge RstN) begin
146     if (!RstN) begin
147         pc <= {`MEM_AW{1'b0}};
148         id_v <= 1'b0;
149         id_ir <= {`IW{1'b0}};
150         ex_v <= 1'b0;
151         ex_is_alu <= 1'b0;
152         ex_writes <= 1'b0;
153         ex_rd <= {`REG_AW{1'b0}};
154         ex_simple <= {`CX_WORD{1'b0}};

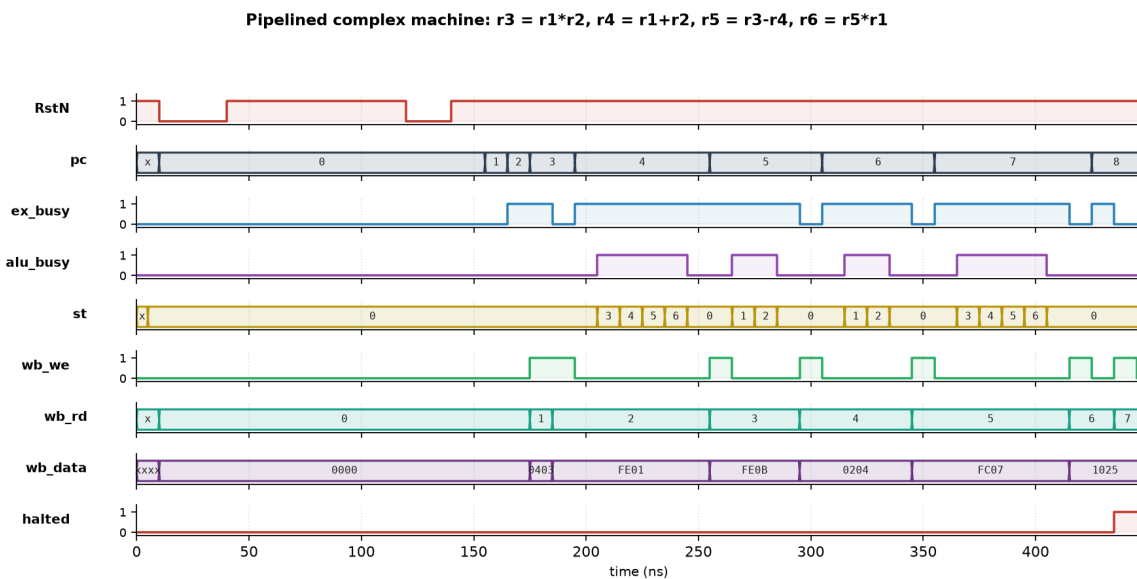
```

```

155     alu_start <= 1'b0;
156     alu_op    <= 2'd0;
157     alu_a     <= {`CX_WORD{1'b0}};
158     alu_b     <= {`CX_WORD{1'b0}};
159     wb_v      <= 1'b0;
160     wb_a      <= {`REG_AW{1'b0}};
161     wb_d      <= {`CX_WORD{1'b0}};
162     halted    <= 1'b0;
163     ovf_sticky <= 1'b0;
164 end else if (run) begin
165     alu_start <= 1'b0;
166     wb_v      <= 1'b0;
167
168     // ---- EX -> WB ----
169     if (ex_completing) begin
170         wb_v <= ex_writes;
171         wb_a <= ex_rd;
172         wb_d <= ex_is_alu ? alu_y : ex_simple;
173         if (ex_is_alu && alu_ovf) ovf_sticky <= 1'b1;
174         ex_v <= 1'b0;
175     end
176
177     // ---- ID -> EX (written after the block above so that an
178     // instruction can enter EX in the very cycle it frees up) ----
179     if (id_go && !id_is_halt) begin
180         ex_v <= 1'b1;
181         ex_is_alu <= id_is_alu;
182         ex_writes <= id_writes;
183         ex_rd <= id_rd;
184         ex_simple <= (id_op == `OP_LDI) ? id_imm : rs_val;
185         if (id_is_alu) begin
186             alu_start <= 1'b1;
187             alu_op <= id_alu_op;
188             alu_a <= rs_val;
189             alu_b <= rt_val;
190         end
191     end
192
193     // ---- IF -> ID ----
194     if (halting) begin
195         halted <= 1'b1;
196         id_v <= 1'b0;
197     end else if (if_go) begin
198         id_v <= 1'b1;
199         id_ir <= if_instr;
200         pc <= pc + 1'b1;
201     end else if (id_go) begin
202         id_v <= 1'b0;
203     end
204 end
205 end
206 endmodule

```


شکل موج



برنامه‌ی نمایش داده‌شده این است:

دستور	محاسبه	مقدار نوشته‌شده
LDI r1, 3+4j	—	0x0403
LDI r2, 1-2j	—	0xFE01
MUL r3, r1, r2	$(3 + 4j)(1 - 2j) = 11 - 2j$	0xFE0B
ADD r4, r1, r2	$(3 + 4j) + (1 - 2j) = 4 + 2j$	0x0204
SUB r5, r3, r4	$(11 - 2j) - (4 + 2j) = 7 - 4j$	0xFC07
MUL r6, r5, r1	$(7 - 4j)(3 + 4j) = 37 + 16j$	0x1025
MOV r7, r6	—	0x1025

سه چیز در شکل موج قابل دیدن است. اول، سیگنال st که حالت داخلی ALU است: برای هر MUL دنباله‌ی ۳، ۴، ۵، ۶ (یعنی M1 تا M4) و برای هر ADD و SUB دنباله‌ی ۱، ۲ دیده می‌شود — همان زمان‌بندی جدول بالا. دوم، alu_busy هرگز دو عملیات را هم‌زمان نشان نمی‌دهد، که همان قید تک‌واحدی است. سوم، pc در حالی جلو می‌رود که ALU هنوز مشغول است: این خود هم‌پوشانی پایپ‌لاین است.

توجه کنید که دو نوشتن آخر (r6 و r7) پشت سر هم و بدون فاصله انجام می‌شوند، چون MOV از ALU استفاده نمی‌کند و بلافاصله پس از آزاد شدن EX وارد آن می‌شود.

تست‌ها

پنج تست وجود دارد که اولین آن‌ها اصلاً شبیه‌سازی نیست.

(۱) **بررسی منبع.** قید «بیش از یک بار استفاده نشود» یک ویژگی متن مدار است، نه یک ویژگی شکل موج. هیچ شبیه‌سازی‌ای نمی‌تواند نشان دهد که دو ضرب‌کننده در مدار وجود ندارد — مداری با دو ضرب‌کننده همان جواب‌های درست را می‌دهد. بنابراین اسکرپت تست پیش از هر شبیه‌سازی، تعداد نمونه‌سازی‌های mul_unit، add_unit و complex_alu را در کل RTL می‌شمارد و اگر هرکدام بیش از یکی باشد تست را رد می‌کند.

(۲ و ۳) **بخش الف و بخش ب.** هر کدام یک مدل مرجع دارند که با حساب صحیح ساده نوشته شده — بدون ماشین حالت و بدون زمان‌بندی — و بنابراین هیچ اشتباهی را با طراحی شریک نمی‌شود. پوشش شامل تمام ترکیب‌های

نه مقدار مرزی روی هر چهار مؤلفه (یعنی $9^4 = 6561$ حالت برای هر عملیات) به‌علاوه‌ی یک جاروب تصادفی است. علاوه بر مقدار عددی، پرچم اشباع و تعداد کلاک هر عملیات هم بررسی می‌شود.

```

1 `timescale 1ns/1ps
2 `include "complex_pkg.vh"
3
4 // Part (a): complex addition and subtraction.
5 //
6 // The DUT is complex_alu in its ADD / SUB modes. Results are compared against
7 // a model written in plain integer arithmetic -- no shift registers, no state
8 // machine, nothing the design and the model could get wrong together.
9 //
10 // Coverage: every combination of nine corner values on each of the four
11 // components (both operations), plus a random sweep, plus a check that the
12 // operation really takes the number of cycles the schedule promises.
13 module tb_complex_addsub;
14     reg          Clk = 1'b0, RstN = 1'b1;
15     reg          start = 1'b0;
16     reg [1:0]    op = 2'd0;
17     reg [`CX_WORD-1:0] a = 16'd0, b = 16'd0;
18     wire [`CX_WORD-1:0] y;
19     wire         ovf, busy, done;
20
21     complex_alu dut (
22         .Clk(Clk), .RstN(RstN), .start(start), .op(op), .a(a), .b(b),
23         .y(y), .ovf(ovf), .busy(busy), .done(done)
24     );
25
26     always #5 Clk = ~Clk;
27
28     integer pass = 0, fail = 0;
29     integer cyc = 0;
30     always @(posedge Clk) cyc = cyc + 1;
31
32     integer i, j, k, m, n, t0, seed;
33     integer ar, ai, br, bi, er, ei;
34     reg     eovf;
35
36     integer corner [0:8];
37
38     function integer sat8i;
39         input integer v;
40         begin
41             if (v > 127) sat8i = 127;
42             else if (v < -128) sat8i = -128;
43             else sat8i = v;
44         end
45     endfunction
46
47     function outside;
48         input integer v;
49         begin
50             outside = (v > 127) || (v < -128);
51         end
52     endfunction
53
54     task automatic chk;
55         input [511:0] name;
56         input         ok;
57         begin
58             if (ok) pass = pass + 1;
59             else begin
60                 fail = fail + 1;
61                 $display("  FAIL @%0t: %0s op=%0d a=%h b=%h y=%h (want re=%0d im=%0d)",
62                     $time, name, op, a, b, y, er, ei);
63             end
64         end
65     endtask
66
67     // Drive one operation and return with y valid.
68     task automatic run_op;
69         input [1:0] o;
70         input [`CX_WORD-1:0] A;
71         input [`CX_WORD-1:0] B;
72         begin
73             @(negedge Clk); op = o; a = A; b = B; start = 1'b1;
74             @(negedge Clk); start = 1'b0; t0 = cyc;
75             while (!done) @(negedge Clk);
76         end
77     endtask
78
79     // Model + compare for one operand pair.
80     task automatic check_pair;
81         input [1:0] o;
82         input [`CX_WORD-1:0] A;
83         input [`CX_WORD-1:0] B;

```

```

84     begin
85         ar = $signed(A[7:0]); ai = $signed(A[15:8]);
86         br = $signed(B[7:0]); bi = $signed(B[15:8]);
87
88         if (o == `ALU_SUB) begin
89             er = sat8i(ar - br); ei = sat8i(ai - bi);
90             eovf = outside(ar - br) || outside(ai - bi);
91         end else begin
92             er = sat8i(ar + br); ei = sat8i(ai + bi);
93             eovf = outside(ar + br) || outside(ai + bi);
94         end
95
96         run_op(o, A, B);
97         chk("real part", y[7:0] === er[7:0]);
98         chk("imaginary part", y[15:8] === ei[7:0]);
99         chk("overflow flag", ovf === eovf);
100     end
101 endtask
102
103 initial begin
104     corner[0] = -128; corner[1] = -127; corner[2] = -64;
105     corner[3] = -1; corner[4] = 0; corner[5] = 1;
106     corner[6] = 5; corner[7] = 64; corner[8] = 127;
107
108     @(negedge Clk); RstN = 1'b0;
109     repeat (3) @(negedge Clk); RstN = 1'b1;
110     @(negedge Clk);
111
112     chk("idle after reset", busy === 1'b0 && done === 1'b0);
113
114     // ---- a worked example, so the report has something concrete ----
115     // (3 + 4j) + (-1 + 2j) = (2 + 6j)
116     run_op(`ALU_ADD, {8'sd4, 8'sd3}, {8'sd2, -8'sd1});
117     chk("(3+4j)+(-1+2j) = 2+6j", y === {8'sd6, 8'sd2});
118     // (3 + 4j) - (-1 + 2j) = (4 + 2j)
119     run_op(`ALU_SUB, {8'sd4, 8'sd3}, {8'sd2, -8'sd1});
120     chk("(3+4j)-(-1+2j) = 4+2j", y === {8'sd2, 8'sd4});
121
122     // ---- latency: exactly two passes through the shared adder ----
123     run_op(`ALU_ADD, {8'sd1, 8'sd1}, {8'sd1, 8'sd1});
124     chk("add occupies the adder for 2 cycles", (cyc - t0) == 2);
125     run_op(`ALU_SUB, {8'sd1, 8'sd1}, {8'sd1, 8'sd1});
126     chk("sub occupies the adder for 2 cycles", (cyc - t0) == 2);
127
128     // ---- every corner combination, both operations ----
129     for (i = 0; i < 9; i = i + 1)
130     for (j = 0; j < 9; j = j + 1)
131     for (k = 0; k < 9; k = k + 1)
132     for (m = 0; m < 9; m = m + 1) begin
133         check_pair(`ALU_ADD, {corner[j][7:0], corner[i][7:0]},
134             {corner[m][7:0], corner[k][7:0]});
135         check_pair(`ALU_SUB, {corner[j][7:0], corner[i][7:0]},
136             {corner[m][7:0], corner[k][7:0]});
137     end
138
139     // ---- random sweep ----
140     seed = 32'd20260826;
141     for (n = 0; n < 1000; n = n + 1) begin
142         check_pair(n[0] ? `ALU_SUB : `ALU_ADD,
143             $random(seed), $random(seed));
144     end
145
146     $display("tb_complex_addsub: Passed: %0d, Failed: %0d", pass, fail);
147     if (fail != 0) $fatal(1);
148     $finish;
149 end
150 endmodule

```

```

1 `timescale 1ns/1ps
2 `include "complex_pkg.vh"
3
4 // Part (b): complex multiplication.
5 //
6 // Same DUT, MUL mode. The model computes
7 // (ar + j*ai)(br + j*bi) = (ar*br - ai*bi) + j(ar*bi + ai*br)
8 // in plain integer arithmetic and then clamps, so it shares nothing with the
9 // four-cycle schedule inside the design.
10 //
11 // Besides the numeric sweep this testbench pins down two structural claims:
12 // the multiply really costs four cycles (one per product), and the ALU is
13 // never asked to start a second operation while the first is running -- which
14 // is the whole point of having a single multiplier.
15 module tb_complex_mul;
16     reg Clk = 1'b0, RstN = 1'b1;
17     reg start = 1'b0;
18     reg [1:0] op = `ALU_MUL;

```

```

19 reg [`CX_WORD-1:0] a = 16'd0, b = 16'd0;
20 wire [`CX_WORD-1:0] y;
21 wire          ovf, busy, done;
22
23 complex_alu dut (
24     .Clk(Clk), .RstN(RstN), .start(start), .op(op), .a(a), .b(b),
25     .y(y), .ovf(ovf), .busy(busy), .done(done)
26 );
27
28 always #5 Clk = ~Clk;
29
30 integer pass = 0, fail = 0;
31 integer cyc = 0;
32 always @(posedge Clk) cyc = cyc + 1;
33
34 // Continuous invariant: a new operation must never be started while the
35 // shared units are still busy with the previous one.
36 always @(posedge Clk)
37     if (RstN && start && busy) begin
38         $display(" FAIL @%0t: start asserted while the ALU is busy", $time);
39         fail = fail + 1;
40     end
41
42 integer i, j, k, m, n, t0, seed;
43 integer ar, ai, br, bi, er, ei, raw_re, raw_im;
44 reg      eovf;
45
46 integer corner [0:8];
47
48 function integer sat8i;
49     input integer v;
50     begin
51         if      (v > 127) sat8i = 127;
52         else if (v < -128) sat8i = -128;
53         else      sat8i = v;
54     end
55 endfunction
56
57 function outside;
58     input integer v;
59     begin
60         outside = (v > 127) || (v < -128);
61     end
62 endfunction
63
64 task automatic chk;
65     input [511:0] name;
66     input      ok;
67     begin
68         if (ok) pass = pass + 1;
69         else begin
70             fail = fail + 1;
71             $display(" FAIL @%0t: %0s a=%h b=%h y=%h (want re=%0d im=%0d)",
72                 $time, name, a, b, y, er, ei);
73         end
74     end
75 endtask
76
77 task automatic run_mul;
78     input [`CX_WORD-1:0] A;
79     input [`CX_WORD-1:0] B;
80     begin
81         @(negedge Clk); a = A; b = B; op = `ALU_MUL; start = 1'b1;
82         @(negedge Clk); start = 1'b0; t0 = cyc;
83         while (!done) @(negedge Clk);
84     end
85 endtask
86
87 task automatic check_pair;
88     input [`CX_WORD-1:0] A;
89     input [`CX_WORD-1:0] B;
90     begin
91         ar = $signed(A[7:0]); ai = $signed(A[15:8]);
92         br = $signed(B[7:0]); bi = $signed(B[15:8]);
93
94         raw_re = ar*br - ai*bi;
95         raw_im = ar*bi + ai*br;
96         er      = sat8i(raw_re);
97         ei      = sat8i(raw_im);
98         eovf     = outside(raw_re) || outside(raw_im);
99
100         run_mul(A, B);
101         chk("real part", y[7:0] === er[7:0]);
102         chk("imaginary part", y[15:8] === ei[7:0]);
103         chk("overflow flag", ovf === eovf);
104     end
105 endtask

```

```

106
107 initial begin
108     corner[0] = -128; corner[1] = -127; corner[2] = -64;
109     corner[3] = -1; corner[4] = 0; corner[5] = 1;
110     corner[6] = 5; corner[7] = 11; corner[8] = 127;
111
112     @(negedge Clk); RstN = 1'b0;
113     repeat (3) @(negedge Clk); RstN = 1'b1;
114     @(negedge Clk);
115
116     // ---- worked examples ----
117     // (3 + 4j)(1 - 2j) = 3 - 6j + 4j + 8 = 11 - 2j
118     run_mul({8'sd4, 8'sd3}, {8'sd2, 8'sd1});
119     chk("(3+4j)(1-2j) = 11-2j", y == {-8'sd2, 8'sd11});
120     // j * j = -1
121     run_mul({8'sd1, 8'sd0}, {8'sd1, 8'sd0});
122     chk("j*j = -1", y == {8'sd0, -8'sd1});
123     // (5 + 0j)(0 + 3j) = 15j
124     run_mul({8'sd0, 8'sd5}, {8'sd3, 8'sd0});
125     chk("5 * 3j = 15j", y == {8'sd15, 8'sd0});
126     // saturation: (100 + 0j)(0 + 100j) = 10000j -> clamped to 127j
127     run_mul({8'sd0, 8'sd100}, {8'sd100, 8'sd0});
128     chk("100 * 100j saturates to 127j", y == {8'sd127, 8'sd0} && ovf == 1'b1);
129
130     // ---- latency: one cycle per real product ----
131     run_mul({8'sd1, 8'sd1}, {8'sd1, 8'sd1});
132     chk("multiply occupies the multiplier for 4 cycles", (cyc - t0) == 4);
133
134     // ---- every corner combination ----
135     for (i = 0; i < 9; i = i + 1)
136     for (j = 0; j < 9; j = j + 1)
137     for (k = 0; k < 9; k = k + 1)
138     for (m = 0; m < 9; m = m + 1)
139         check_pair({corner[j][7:0], corner[i][7:0]},
140                     {corner[m][7:0], corner[k][7:0]});
141
142     // ---- random sweep ----
143     seed = 32'd1405;
144     for (n = 0; n < 1000; n = n + 1)
145         check_pair($random(seed), $random(seed));
146
147     $display("tb_complex_mul: Passed: %0d, Failed: %0d", pass, fail);
148     if (fail != 0) $fatal(1);
149     $finish;
150 end
151 endmodule

```

۴) ماشین در برابر مدل مرجع. مدل یک مفسر ساده‌ی همان برنامه است: بدون پایپ‌لاین، بدون توقف و بدون واحد مشترک. توافق با آن یعنی قفل‌ها و مسیر ارسال به جلو دقیقاً همان نتیجه‌ای را می‌سازند که اجرای دستورات یکی یکی می‌ساخت.

بیشتر کار را برنامه‌های تصادفی انجام می‌دهند. برنامه‌ی دست‌نویس فقط الگوهای وابستگی‌ای را پوشش می‌دهد که نویسنده‌اش به آن‌ها فکر کرده؛ صد و پنجاه برنامه‌ی تصادفی زنجیره‌های وابستگی، استفاده‌ی مجدد از ثبات‌ها و هم‌نامی عملوندها را پوشش می‌دهند که هیچ‌کس آن‌ها را عمداً نمی‌نویسد.

```

1 `timescale 1ns/1ps
2 `include "complex_pkg.vh"
3
4 // Part (c): the pipelined machine, compared against a reference model.
5 //
6 // The model is a plain sequential interpreter of the same program: it has no
7 // pipeline, no stalls and no shared units, so agreeing with it means the
8 // pipeline's interlocks and forwarding produce exactly the result of running
9 // the instructions one at a time.
10 //
11 // Most of the work is done by RANDOM PROGRAMS. Hand-written programs only
12 // cover the hazard patterns their author thought of; a few hundred random ones
13 // cover dependency chains, register reuse and operand aliasing that nobody
14 // would think to write down.
15 module tb_complex_cpu;
16     reg Clk = 1'b0, RstN = 1'b1, run_en = 1'b0;
17     reg prog_we = 1'b0;
18     reg [`MEM_AW-1:0] prog_addr = 5'd0;
19     reg [`IW-1:0] prog_data = 32'd0;
20     reg [`REG_AW-1:0] dbg_raddr = 3'd0;
21
22     wire [`CX_WORD-1:0] dbg_rdata;
23     wire [`MEM_AW-1:0] pc;
24     wire halted, finished, ovf_sticky, alu_busy, ex_busy, wb_we;
25     wire [`REG_AW-1:0] wb_rd;
26     wire [`CX_WORD-1:0] wb_data;

```

```

27
28 complex_cpu dut (
29     .Clk(Clk), .RstN(RstN), .run(run_en),
30     .prog_we(prog_we), .prog_addr(prog_addr), .prog_data(prog_data),
31     .dbg_raddr(dbg_raddr), .dbg_rdata(dbg_rdata),
32     .pc(pc), .halted(halted), .finished(finished), .ovf_sticky(ovf_sticky),
33     .alu_busy(alu_busy), .ex_busy(ex_busy),
34     .wb_we(wb_we), .wb_rd(wb_rd), .wb_data(wb_data)
35 );
36
37 always #5 Clk = ~Clk;
38
39 integer pass = 0, fail = 0;
40 integer i, n, prog_len, seed, guard;
41
42 reg [`IW-1:0]      prog [0:`MEM_WORDS-1];
43 reg [`CX_WORD-1:0] mreg [0:7];      // model register file
44 reg               movf;              // model sticky overflow
45
46 task automatic chk;
47     input [511:0] name;
48     input      ok;
49     begin
50         if (ok) pass = pass + 1;
51         else begin
52             fail = fail + 1;
53             $display(" FAIL @%0t: %0s", $time, name);
54         end
55     end
56 endtask
57
58 function [`IW-1:0] enc;
59     input [3:0]      o;
60     input [`REG_AW-1:0] rd;
61     input [`REG_AW-1:0] rs;
62     input [`REG_AW-1:0] rt;
63     input [`CX_WORD-1:0] imm;
64     begin
65         enc = {o, rd, rs, rt, 3'b000, imm};
66     end
67 endfunction
68
69 function integer sat8i;
70     input integer v;
71     begin
72         if      (v > 127) sat8i = 127;
73         else if (v < -128) sat8i = -128;
74         else          sat8i = v;
75     end
76 endfunction
77
78 function outside;
79     input integer v;
80     begin
81         outside = (v > 127) || (v < -128);
82     end
83 endfunction
84
85 // ---- the reference model: run the program one instruction at a time ----
86 task automatic model_run;
87     integer p, ar, ai, br, bi, rr, ri, rr8, ri8;
88     reg [`IW-1:0]      ins;
89     reg [3:0]          o;
90     reg [`REG_AW-1:0]  rd, rs, rt;
91     integer            g;
92     begin
93         for (g = 0; g < 8; g = g + 1) mreg[g] = {'CX_WORD{1'b0}};
94         movf = 1'b0;
95         p = 0;
96         g = 0;
97         while (p < `MEM_WORDS && g < 4*`MEM_WORDS) begin
98             g = g + 1;
99             ins = prog[p];
100             o = ins[31:28];
101             rd = ins[27:25];
102             rs = ins[24:22];
103             rt = ins[21:19];
104
105             ar = $signed(mreg[rs][7:0]);  ai = $signed(mreg[rs][15:8]);
106             br = $signed(mreg[rt][7:0]);  bi = $signed(mreg[rt][15:8]);
107
108             case (o)
109                 `OP_HALT: p = `MEM_WORDS;      // stop
110                 `OP_NOP:  p = p + 1;
111                 `OP_LDI:  begin mreg[rd] = ins[15:0];  p = p + 1; end
112                 `OP_MOV:  begin mreg[rd] = mreg[rs];  p = p + 1; end
113                 `OP_ADD, `OP_SUB, `OP_MUL: begin

```

```

114         if (o == `OP_ADD) begin rr = ar + br;      ri = ai + bi;      end
115         else if (o == `OP_SUB) begin rr = ar - br;  ri = ai - bi;      end
116         else begin rr = ar*br - ai*bi;             ri = ar*bi + ai*br; end
117         if (outside(rr) || outside(ri)) movf = 1'b1;
118         rr8 = sat8i(rr);
119         ri8 = sat8i(ri);
120         mreg[rd] = {ri8[7:0], rr8[7:0]};
121         p = p + 1;
122     end
123     default: p = p + 1;          // unused opcode acts as NOP
124 endcase
125 end
126 end
127 endtask
128
129 // ---- load a program, reset the pipeline, run it to completion ----
130 task automatic hw_run;
131     output integer cycles;
132     integer c;
133     begin
134         run_en = 1'b0;
135         @(negedge Clk);
136         for (c = 0; c < `MEM_WORDS; c = c + 1) begin
137             prog_we = 1'b1; prog_addr = c[`MEM_AW-1:0]; prog_data = prog[c];
138             @(negedge Clk);
139         end
140         prog_we = 1'b0;
141
142         RstN = 1'b0; @(negedge Clk); @(negedge Clk); RstN = 1'b1;
143         @(negedge Clk);
144
145         run_en = 1'b1;
146         c = 0;
147         // `finished` (not `halted`) is the safe point to read results:
148         // it also requires the last write-back to have landed.
149         while (!finished && c < 4000) begin
150             @(negedge Clk);
151             c = c + 1;
152         end
153         cycles = c;
154         run_en = 1'b0;
155     end
156 endtask
157
158 task automatic compare_state;
159     input [511:0] tag;
160     integer g;
161     reg [`CX_WORD-1:0] got;
162     begin
163         for (g = 0; g < 8; g = g + 1) begin
164             dbg_raddr = g[`REG_AW-1:0]; #1; got = dbg_rdata;
165             if (got != mreg[g]) begin
166                 fail = fail + 1;
167                 $display(" FAIL @%0t: %0s r%0d = %h, model says %h",
168                     $time, tag, g, got, mreg[g]);
169             end else pass = pass + 1;
170         end
171         chk("sticky overflow flag", ovf_sticky == movf);
172     end
173 endtask
174
175 task automatic rd_hw;
176     input [`REG_AW-1:0] r;
177     output [`CX_WORD-1:0] v;
178     begin
179         dbg_raddr = r; #1; v = dbg_rdata;
180     end
181 endtask
182
183 task automatic fill_halt;
184     integer g;
185     begin
186         for (g = 0; g < `MEM_WORDS; g = g + 1)
187             prog[g] = enc(`OP_HALT, 3'd0, 3'd0, 3'd0, 16'd0);
188     end
189 endtask
190
191 integer cycles;
192 reg [`CX_WORD-1:0] tmp;
193
194 initial begin
195     @(negedge Clk); RstN = 1'b0;
196     repeat (3) @(negedge Clk); RstN = 1'b1;
197
198     // ----- a demo program --
199     // r1 = 3 + 4j ; r2 = 1 - 2j
200     // r3 = r1 * r2 = 11 - 2j

```

```

201 // r4 = r1 + r2 = 4 + 2j
202 // r5 = r3 - r4 = 7 - 4j
203 // r6 = r5 * r1 = 37 + 16j
204 fill_halt;
205 prog[0] = enc(`OP_LDI, 3'd1, 3'd0, 3'd0, {8'sd4, 8'sd3});
206 prog[1] = enc(`OP_LDI, 3'd2, 3'd0, 3'd0, {-8'sd2, 8'sd1});
207 prog[2] = enc(`OP_MUL, 3'd3, 3'd1, 3'd2, 16'd0);
208 prog[3] = enc(`OP_ADD, 3'd4, 3'd1, 3'd2, 16'd0);
209 prog[4] = enc(`OP_SUB, 3'd5, 3'd3, 3'd4, 16'd0);
210 prog[5] = enc(`OP_MUL, 3'd6, 3'd5, 3'd1, 16'd0);
211 prog[6] = enc(`OP_MOV, 3'd7, 3'd6, 3'd0, 16'd0);
212 prog[7] = enc(`OP_HALT, 3'd0, 3'd0, 3'd0, 16'd0);
213
214 model_run;
215 hw_run(cycles);
216 chk("demo program halted", halted === 1'b1);
217 compare_state("demo program");
218 rd_hw(3'd3, tmp); chk("demo: r3 = 11 - 2j", tmp === {-8'sd2, 8'sd11});
219 rd_hw(3'd5, tmp); chk("demo: r5 = 7 - 4j", tmp === {-8'sd4, 8'sd7});
220 rd_hw(3'd6, tmp); chk("demo: r6 = 37 + 16j", tmp === {8'sd16, 8'sd37});
221 $display(" demo program: %0d instructions in %0d cycles", 8, cycles);
222
223 // ----- a dependency chain: every result feeds the next --
224 fill_halt;
225 prog[0] = enc(`OP_LDI, 3'd1, 3'd0, 3'd0, {8'sd1, 8'sd2});
226 prog[1] = enc(`OP_LDI, 3'd2, 3'd0, 3'd0, {8'sd1, 8'sd1});
227 prog[2] = enc(`OP_ADD, 3'd3, 3'd1, 3'd2, 16'd0);
228 prog[3] = enc(`OP_ADD, 3'd3, 3'd3, 3'd2, 16'd0);
229 prog[4] = enc(`OP_ADD, 3'd3, 3'd3, 3'd2, 16'd0);
230 prog[5] = enc(`OP_MUL, 3'd3, 3'd3, 3'd2, 16'd0);
231 prog[6] = enc(`OP_SUB, 3'd3, 3'd3, 3'd1, 16'd0);
232 prog[7] = enc(`OP_HALT, 3'd0, 3'd0, 3'd0, 16'd0);
233 model_run;
234 hw_run(cycles);
235 chk("chain program halted", halted === 1'b1);
236 compare_state("back-to-back dependency chain");
237
238 // ----- saturation propagates through a program --
239 fill_halt;
240 prog[0] = enc(`OP_LDI, 3'd1, 3'd0, 3'd0, {8'sd100, 8'sd100});
241 prog[1] = enc(`OP_LDI, 3'd2, 3'd0, 3'd0, {8'sd100, 8'sd100});
242 prog[2] = enc(`OP_ADD, 3'd3, 3'd1, 3'd2, 16'd0);
243 prog[3] = enc(`OP_MUL, 3'd4, 3'd1, 3'd2, 16'd0);
244 prog[4] = enc(`OP_HALT, 3'd0, 3'd0, 3'd0, 16'd0);
245 model_run;
246 hw_run(cycles);
247 compare_state("saturating program");
248 chk("overflow was reported", ovf_sticky === 1'b1);
249
250 // ----- random programs --
251 seed = 32'd8080;
252 for (n = 0; n < 150; n = n + 1) begin
253   fill_halt;
254   prog_len = 4 + ({$random(seed)} % 20);
255   for (i = 0; i < prog_len; i = i + 1) begin
256     case ({$random(seed)} % 8)
257       0, 1: prog[i] = enc(`OP_LDI, ({$random(seed)} % 8), 3'd0, 3'd0,
258                               {$random(seed)});
259       2:   prog[i] = enc(`OP_MOV, ({$random(seed)} % 8),
260                               ({$random(seed)} % 8), 3'd0, 16'd0);
261       3:   prog[i] = enc(`OP_NOP, 3'd0, 3'd0, 3'd0, 16'd0);
262       4, 5: prog[i] = enc(`OP_ADD, ({$random(seed)} % 8),
263                               ({$random(seed)} % 8), ({$random(seed)} % 8), 16'd0);
264       6:   prog[i] = enc(`OP_SUB, ({$random(seed)} % 8),
265                               ({$random(seed)} % 8), ({$random(seed)} % 8), 16'd0);
266       7:   prog[i] = enc(`OP_MUL, ({$random(seed)} % 8),
267                               ({$random(seed)} % 8), ({$random(seed)} % 8), 16'd0);
268     endcase
269   end
270   prog[prog_len] = enc(`OP_HALT, 3'd0, 3'd0, 3'd0, 16'd0);
271
272   model_run;
273   hw_run(cycles);
274   chk("random program halted", halted === 1'b1);
275   compare_state("random program");
276 end
277
278 $display("tb_complex_cpu: Passed: %0d, Failed: %0d", pass, fail);
279 if (fail != 0) $fatal(1);
280 $finish;
281 end
282 endmodule

```

(۵) ادعاهای ساختاری پایپ‌لاین. مقایسه با مدل مرجع نشان می‌دهد ماشین عدد درست را می‌دهد، ولی دربارهی

اینکه چرا این قدر طول می‌کشد چیزی نمی‌گوید — ماشینی که بی‌سروصدا هر دستور را جداگانه و بدون هم‌پوشانی اجرا کند هم همان تست را پاس می‌کند. تست پنجم دقیقاً همین را اندازه می‌گیرد:

سناریو	کلاک	نتیجه‌گیری
۱۶ دستور تک‌کلاکی مستقل	۱۹	حدود یک کلاک برای هر دستور؛ بدون هم‌پوشانی دست‌کم 16×3 لازم بود
۱۶ ضرب مستقل	۱۰۲	گلوگاه، ضرب‌کننده‌ی مشترک است نه پایپ‌لاین
۱۶ ضرب زنجیره‌ای وابسته	۱۱۷	هزینه‌ی قفل داده

سطر آخر عدد قشنگی می‌دهد: اختلاف $15 = 102 - 117$ کلاک است و تعداد جفت‌های وابسته هم دقیقاً ۱۵ تاست. یعنی قفل داده دقیقاً یک کلاک برای هر وابستگی هزینه دارد — همان یک کلاکی که مصرف‌کننده منتظر می‌ماند تا تولیدکننده از EX به WB برود و مسیر ارسال به جلو در دسترس شود. این تساوی در تست‌بنچ به‌صورت یک شرط دقیق نوشته شده، نه به‌صورت یک بازه.

```

1 `timescale 1ns/1ps
2 `include "complex_pkg.vh"
3
4 // Structural checks on the machine, aimed at the claims the reference-model
5 // testbench cannot make.
6 //
7 // A model comparison proves the machine computes the right numbers. It says
8 // nothing about WHY it takes the time it takes, and a machine that quietly
9 // executed every instruction one at a time would pass it just as well. The
10 // checks here are about the pipeline itself:
11 //
12 // * instructions really do overlap -- a run of independent single-cycle
13 //   instructions costs about one cycle each, which is impossible without
14 //   fetch, decode and write-back running underneath execute;
15 // * the shared ALU really is the throughput limit for arithmetic;
16 // * the interlock costs exactly as much as it must and no more;
17 // * nothing is executed after HALT, and nothing is written twice.
18 module tb_complex_cpu_spec;
19     reg          Clk = 1'b0, RstN = 1'b1, run_en = 1'b0;
20     reg          prog_we = 1'b0;
21     reg [MEM_AW-1:0] prog_addr = 5'd0;
22     reg [IW-1:0]   prog_data = 32'd0;
23     reg [REG_AW-1:0] dbg_raddr = 3'd0;
24
25     wire [CX_WORD-1:0] dbg_rdata;
26     wire [MEM_AW-1:0] pc;
27     wire halted, finished, ovf_sticky, alu_busy, ex_busy, wb_we;
28     wire [REG_AW-1:0] wb_rd;
29     wire [CX_WORD-1:0] wb_data;
30
31     complex_cpu dut (
32         .Clk(Clk), .RstN(RstN), .run(run_en),
33         .prog_we(prog_we), .prog_addr(prog_addr), .prog_data(prog_data),
34         .dbg_raddr(dbg_raddr), .dbg_rdata(dbg_rdata),
35         .pc(pc), .halted(halted), .finished(finished), .ovf_sticky(ovf_sticky),
36         .alu_busy(alu_busy), .ex_busy(ex_busy),
37         .wb_we(wb_we), .wb_rd(wb_rd), .wb_data(wb_data)
38     );
39
40     always #5 Clk = ~Clk;
41
42     integer pass = 0, fail = 0;
43     integer i, writes, cyc_ldi, cyc_mul_indep, cyc_mul_dep, cyc_nop;
44     reg [IW-1:0] prog [0:MEM_WORDS-1];
45     reg [CX_WORD-1:0] tmp;
46
47     // Count every register write the machine performs.
48     integer wcount = 0;
49     always @(posedge Clk) if (run_en && wb_we) wcount = wcount + 1;
50
51     // The ALU is a single shared resource: it must never be busy while the
52     // execute stage claims to be empty, and vice versa a busy ALU must always
53     // belong to the one instruction sitting in EX.
54     always @(posedge Clk)
55         if (RstN && run_en && alu_busy && !ex_busy) begin
56             $display(" FAIL %0t: the ALU is running with no instruction in EX", $time);
57             fail = fail + 1;
58         end
59

```

```

60 task automatic chk;
61     input [511:0] name;
62     input      ok;
63     begin
64         if (ok) pass = pass + 1;
65         else begin
66             fail = fail + 1;
67             $display(" FAIL @%0t: %0s", $time, name);
68         end
69     end
70 endtask
71
72 function [1:0] enc;
73     input [3:0] o;
74     input [1:0] rd;
75     input [1:0] rs;
76     input [1:0] rt;
77     input [1:0] imm;
78     begin
79         enc = {o, rd, rs, rt, 3'b000, imm};
80     end
81 endfunction
82
83 task automatic fill_halt;
84     integer g;
85     begin
86         for (g = 0; g < `MEM_WORDS; g = g + 1)
87             prog[g] = enc(`OP_HALT, 3'd0, 3'd0, 3'd0, 16'd0);
88     end
89 endtask
90
91 task automatic hw_run;
92     output integer cycles;
93     integer c;
94     begin
95         run_en = 1'b0;
96         @(negedge Clk);
97         for (c = 0; c < `MEM_WORDS; c = c + 1) begin
98             prog_we = 1'b1; prog_addr = c[1:0]; prog_data = prog[c];
99             @(negedge Clk);
100         end
101         prog_we = 1'b0;
102
103         RstN = 1'b0; @(negedge Clk); @(negedge Clk); RstN = 1'b1;
104         @(negedge Clk);
105
106         wcount = 0;
107         run_en = 1'b1;
108         c = 0;
109         // `finished` (not `halted`) is the safe point to read results:
110         // it also requires the last write-back to have landed.
111         while (!finished && c < 4000) begin
112             @(negedge Clk);
113             c = c + 1;
114         end
115         cycles = c;
116         run_en = 1'b0;
117     end
118 endtask
119
120 task automatic rd_hw;
121     input [1:0] r;
122     output [1:0] v;
123     begin
124         dbg_raddr = r; #1; v = dbg_rdata;
125     end
126 endtask
127
128 initial begin
129     @(negedge Clk); RstN = 1'b0;
130     repeat (3) @(negedge Clk); RstN = 1'b1;
131
132     // -----
133     // A) 16 independent single-cycle instructions.
134     // Without overlap this would cost at least 3 cycles each (fetch,
135     // decode, execute). Coming in near 1 cycle each is the pipeline.
136     // -----
137     fill_halt;
138     for (i = 0; i < 16; i = i + 1)
139         prog[i] = enc(`OP_LDI, i[2:0], 3'd0, 3'd0, {8'd0, i[7:0]});
140     prog[16] = enc(`OP_HALT, 3'd0, 3'd0, 3'd0, 16'd0);
141     hw_run(cyc_ldi);
142     $display(" 16 independent single-cycle instructions: %0d cycles", cyc_ldi);
143     chk("independent instructions overlap", cyc_ldi < 2*16);
144     chk("all 16 results were written", wcount == 16);
145     rd_hw(3'd7, tmp);
146     chk("last LDI landed", tmp == {8'd0, 8'd15});

```

```

147 // -----
148 // B) 16 independent multiplies. Each one needs the single multiplier
149 //     for four cycles, so this is the cost of the resource constraint,
150 //     not of the pipeline.
151 // -----
152 fill_halt;
153 prog[0] = enc(OP_LDI, 3'd1, 3'd0, 3'd0, {8'sd1, 8'sd2});
154 prog[1] = enc(OP_LDI, 3'd2, 3'd0, 3'd0, {8'sd1, 8'sd1});
155 for (i = 0; i < 16; i = i + 1)
156     prog[2+i] = enc(OP_MUL, 3'd3 + (i % 5), 3'd1, 3'd2, 16'd0);
157 prog[18] = enc(OP_HALT, 3'd0, 3'd0, 3'd0, 16'd0);
158 hw_run(cyc_mul_indep);
159 $display(" 16 independent multiplies:           %0d cycles", cyc_mul_indep);
160 chk("multiplies are limited by the shared multiplier", cyc_mul_indep > 4*16);
161 chk("all 18 results were written", wcount == 18);
162
163 // -----
164 // C) the same 16 multiplies, but each one consuming the previous
165 //     result. The interlock must add stalls on top of B.
166 // -----
167 fill_halt;
168 prog[0] = enc(OP_LDI, 3'd1, 3'd0, 3'd0, {8'sd0, 8'sd1});
169 prog[1] = enc(OP_LDI, 3'd2, 3'd0, 3'd0, {8'sd1, 8'sd1});
170 for (i = 0; i < 16; i = i + 1)
171     prog[2+i] = enc(OP_MUL, 3'd2, 3'd2, 3'd1, 16'd0);
172 prog[18] = enc(OP_HALT, 3'd0, 3'd0, 3'd0, 16'd0);
173 hw_run(cyc_mul_dep);
174 $display(" 16 dependent multiplies:           %0d cycles", cyc_mul_dep);
175 chk("a dependency chain costs more than independent work",
176     cyc_mul_dep > cyc_mul_indep);
177 // 16 multiplies form 15 dependent pairs, and the interlock costs
178 // exactly one cycle for each: the consumer waits in decode for the
179 // cycle in which the producer moves from execute to write-back, and
180 // is then served by the forward.
181 $display(" cost of the interlock: %0d cycles over %0d dependent pairs",
182     cyc_mul_dep - cyc_mul_indep, 15);
183 chk("the interlock costs exactly one cycle per dependent pair",
184     (cyc_mul_dep - cyc_mul_indep) == 15);
185 // r1 = 1 (real), so multiplying by it repeatedly must not change r2
186 rd_hw(3'd2, tmp);
187 chk("x * 1 repeated 16 times is still x", tmp == {8'sd1, 8'sd1});
188
189 // -----
190 // D) NOPs are real bubbles: they occupy the pipeline but write nothing.
191 // -----
192 fill_halt;
193 prog[0] = enc(OP_LDI, 3'd1, 3'd0, 3'd0, {8'sd7, 8'sd7});
194 for (i = 1; i < 12; i = i + 1)
195     prog[i] = enc(OP_NOP, 3'd0, 3'd0, 3'd0, 16'd0);
196 prog[12] = enc(OP_HALT, 3'd0, 3'd0, 3'd0, 16'd0);
197 hw_run(cyc_nop);
198 chk("a NOP writes no register", wcount == 1);
199 rd_hw(3'd1, tmp);
200 chk("the LDI before the NOPs still landed", tmp == {8'sd7, 8'sd7});
201
202 // -----
203 // E) HALT really stops the machine: nothing after it is fetched.
204 // -----
205 fill_halt;
206 prog[0] = enc(OP_LDI, 3'd1, 3'd0, 3'd0, {8'sd3, 8'sd3});
207 prog[1] = enc(OP_HALT, 3'd0, 3'd0, 3'd0, 16'd0);
208 prog[2] = enc(OP_LDI, 3'd2, 3'd0, 3'd0, {8'sd9, 8'sd9});
209 prog[3] = enc(OP_LDI, 3'd3, 3'd0, 3'd0, {8'sd9, 8'sd9});
210 hw_run(i);
211 chk("machine halted", halted == 1'b1);
212 chk("only the instruction before HALT wrote", wcount == 1);
213 rd_hw(3'd1, tmp); chk("r1 was written", tmp == {8'sd3, 8'sd3});
214 rd_hw(3'd2, tmp); chk("r2 after HALT untouched", tmp == 16'd0);
215 rd_hw(3'd3, tmp); chk("r3 after HALT untouched", tmp == 16'd0);
216
217 // the program counter must stay put once halted
218 writes = pc;
219 repeat (10) @(negedge Clk);
220 chk("PC frozen after HALT", pc == writes[MEM_AW-1:0]);
221
222 // -----
223 // F) an operand that is also the destination must still read the OLD
224 //     value: rd is written only at write-back.
225 // -----
226 fill_halt;
227 prog[0] = enc(OP_LDI, 3'd1, 3'd0, 3'd0, {8'sd0, 8'sd3}); // 3
228 prog[1] = enc(OP_ADD, 3'd1, 3'd1, 3'd1, 16'd0); // 6
229 prog[2] = enc(OP_ADD, 3'd1, 3'd1, 3'd1, 16'd0); // 12
230 prog[3] = enc(OP_MUL, 3'd1, 3'd1, 3'd1, 16'd0); // 144 -> clamps
231 prog[4] = enc(OP_HALT, 3'd0, 3'd0, 3'd0, 16'd0);
232 hw_run(i);
233

```

```

234 rd_hw(3'd1, tmp);
235 chk("rd used as both sources reads the old value",
236     tmp == {8'sd0, 8'sd127});
237 chk("the clamp was reported", ovf_sticky == 1'b1);
238
239 $display("tb_complex_cpu_spec: Passed: %0d, Failed: %0d", pass, fail);
240 if (fail != 0) $fatal(1);
241 $finish;
242
243 end
endmodule

```

نتایج تست

```

=== resource check (one adder, one multiplier) ===
mul_unit instantiated 1 time
add_unit instantiated 1 time
complex_alu instantiated 1 time
PASS: resource check

iverilog: Icarus Verilog version 12.0 (stable) ()

=== tb_complex_addsub ===
tb_complex_addsub: Passed: 42371, Failed: 0
PASS: tb_complex_addsub

=== tb_complex_mul ===
tb_complex_mul: Passed: 22688, Failed: 0
PASS: tb_complex_mul

=== tb_complex_cpu ===
demo program: 8 instructions in 30 cycles
tb_complex_cpu: Passed: 1533, Failed: 0
PASS: tb_complex_cpu

=== tb_complex_cpu_spec ===
16 independent single-cycle instructions: 19 cycles
16 independent multiplies: 102 cycles
16 dependent multiplies: 117 cycles
cost of the interlock: 15 cycles over 15 dependent pairs
tb_complex_cpu_spec: Passed: 18, Failed: 0
PASS: tb_complex_cpu_spec

summary: 5/5 passed, 0 failed

```

جمع‌بندی

درس اصلی این آزمایش این است که یک قید منابع، معماری را عوض می‌کند نه فقط اندازه‌اش را. با اجازه‌ی داشتن چهار ضرب‌کننده، ضرب مختلط یک مدار ترکیبی یک‌کلاکی می‌شد و مرحله‌ی اجرا هم مثل بقیه‌ی مراحل پایپ‌لاین می‌شد. با یک ضرب‌کننده، همان عملیات به یک ماشین حالت چهارکلاکی تبدیل می‌شود و در نتیجه پایپ‌لاین به قفل و توقف نیاز پیدا می‌کند — چیزی که بدون آن قید اصلاً لازم نبود.

نکته‌ی دوم درباره‌ی تست است. چهار تست‌بنچ این آزمایش هیچ‌کدام نمی‌توانند نشان دهند که قید اصلی رعایت شده است، چون مداری با دو ضرب‌کننده همان جواب‌های درست را در زمان کمتری می‌دهد و همه‌ی آن‌ها را پاس می‌کند. برای ویژگی‌هایی که درباره‌ی ساختار مدارند و نه رفتار آن، باید سراغ خود متن رفت؛ به همین دلیل شمارش نمونه‌سازی‌ها به‌عنوان اولین تست به اسکرپت اضافه شد.